

```

import numpy as np

# -----
# Part 1: 0/1 Knapsack Problem
# -----
def knapsack(profits, weights, capacity):
    n = len(profits)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    # Build DP table
    for i in range(1, n + 1):
        for w in range(1, capacity + 1):
            if weights[i-1] <= w:
                dp[i][w] = max(profits[i-1] + dp[i-1][w - weights[i-1]], dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]

    # Backtrack to find chosen items
    chosen_items = []
    w = capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i-1][w]:
            chosen_items.append(i-1)
            w -= weights[i-1]

    return dp[n][capacity], list(reversed(chosen_items))

# Example Knapsack Data
profits = [2, 4, 7, 10]
weights = [1, 3, 5, 7]
capacity = 8

max_profit, chosen = knapsack(profits, weights, capacity)
print("==== 0/1 Knapsack ====")
print("Profits:", profits)
print("Weights:", weights)
print("Capacity:", capacity)
print("Max Profit:", max_profit)
print("Chosen Items (indices):", chosen)
print("Chosen Items (profits):", [profits[i] for i in chosen])

# -----
# Part 2: Ridge Regression
# -----
def ridge_regression(X, y, lam):
    """Closed-form solution of Ridge Regression"""
    n_features = X.shape[1]

```

```

w = np.linalg.inv(X.T @ X + lam * np.eye(n_features)) @ X.T @ y
return w

# Toy dataset
X = np.array([[1,1],
              [1,2],
              [1,3]]) # feature matrix
y = np.array([1,2,3]) # target
lam = 1           # regularization parameter

w = ridge_regression(X, y, lam)
y_pred = X @ w

print("\n===== Ridge Regression =====")
print("Optimal Parameters (w):", w)
print("Predictions:", y_pred)
print("True y:", y)

# -----
# Conceptual Link
# -----
print("\n===== Connection =====")
print("Knapsack: trade-off between profit vs weight (constraint)")
print("Ridge Regression: trade-off between error reduction vs parameter penalty (regularization)")

```

Output:

```

> whoami
onkar
> python3 code.py
===== 0/1 Knapsack =====
Profits: [2, 4, 7, 10]
Weights: [1, 3, 5, 7]
Capacity: 8
Max Profit: 12
Chosen Items (indices): [0, 3]
Chosen Items (profits): [2, 10]

===== Ridge Regression =====
Optimal Parameters (w): [0.25      0.83333333]
Predictions: [1.08333333 1.91666667 2.75      ]
True y: [1 2 3]

===== Connection =====
Knapsack: trade-off between profit vs weight (constraint)
Ridge Regression: trade-off between error reduction vs parameter penalty (regularization)

~/Practical/SEM V/DAA/practical3 main* >

```

